

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- Compute n and $\phi(n)$
- Choose $e=7$ and find the private key d
- Encrypt the message $M=9$
- Decrypt the ciphertext and verify the result
- Analyze how changing e affects security

2. Block Cipher Mode Evaluation

Given plaintext blocks:

$P=[1010,1010,1111,1010]$, Key = $K=1100$

- a. Encrypt using ECB and CBC modes (Assume IV = 0000)
- b. Compare the ciphertext outputs
- c. Analyze which mode hides patterns better and explain why

3. DES vs 3-DES vs Blowfish Performance Study

Given:

- a. DES key size = 56 bits, time = 2 ms/block
- b. 3-DES key size = 168 bits, time = 6 ms/block
- c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

- i. Calculate total encryption time for each algorithm
- ii. Analyze trade-offs between security and performance
- iii. Recommend the best algorithm for secure real-time communication

4. Diffie-Hellman Key Exchange Analysis

Given:

- a. $p=23$, $g=5$
- b. User A private key = 6
- c. User B private key = 15
- d. Compute public keys for A and B
- e. Compute the shared secret key
- f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange
- g. Suggest a secure method to prevent the attack

5. ECC vs RSA Comparison

Given:

- a. RSA key size = 1024 bits
- b. ECC key size = 160 bits (equivalent security)
- c. Device processing capacity = limited (IoT device)
- d. Analyze computational cost for both systems
- e. Estimate which consumes less power and memory
- f. Justify which cryptosystem is better for the given scenario with reasoning

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

strong
justification

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWERS – ANALYTICAL PROBLEM SOLVING

Q1. RSA Encryption Analysis

Given

- $p=11$
- $q=13$
- $e=7$
- Message $M=9$

a. Compute n and $\phi(n)$

RSA modulus:

$$\begin{aligned}n &= p \times q \\n &= 11 \times 13 \\n &= 143\end{aligned}$$

Euler's Totient Function:

$$\begin{aligned}\phi(n) &= (p-1)(q-1) \\ \phi(n) &= (11-1)(13-1) \\ \phi(n) &= 10 \times 12 \\ \phi(n) &= 120\end{aligned}$$

Therefore,

$$n=143, \phi(n)=120$$

b. Find the Private Key d

Given:

$$e=7$$

The private exponent d must satisfy:

$$e \times d \equiv 1 \pmod{\phi(n)}$$

Therefore,

$$7d \equiv 1 \pmod{120}$$

We need a value of d such that:

$$7d = 1 + 120k$$

Taking $k=1$:

$$d = \frac{121}{7}$$

which is not an integer.

Taking $k=4$:

$$d = \frac{1 + (120 \times 4)}{7} = \frac{481}{7} = 68.714...$$

This shows an important problem with the given value $e=7$:

$$\gcd(7, 120) = 1$$

so an inverse must exist. Using the Extended Euclidean Algorithm:

$$120 = 7(17) + 1$$

Therefore,

$$1 = 120 - 7(17)$$

Thus:

$$-17(7) \equiv 1 \pmod{120} \quad d = -17 \pmod{120} = 103$$

Verification:

$$7 \times 103 = 721 \quad 721 \pmod{120} = 1$$

Hence:

Public Key = (7, 143) Private Key = (103, 143)

c. Encrypt Message $M=9$

RSA encryption formula:

$$C = M \bmod n \quad C = M^e \bmod n \quad C = M \bmod n \quad C = 97 \bmod 143 \quad C = 9^7 \bmod 143 \quad C = 97 \bmod 143$$

Using modular exponentiation:

$$92 = 81 \times 9^2 = 81 \times 9^2 = 81 \times 81 = 6561 \quad 94 = 81 \times 2 = 6561 \times 9^4 = 81^2 \times 2 = 6561 \times 2 = 1266561 \quad 94 = 81 \times 2 = 6561$$

$$6561 \bmod 143 = 126 \quad 6561 \bmod 143 = 126 \quad 6561 \bmod 143 = 126$$

Therefore:

$$97 = 94 \times 92 \times 99^7 = 9^4 \times 9^2 \times 99^7 = 94 \times 92 \times 9$$

$$C = (126 \times 81 \times 9) \bmod 143 \quad C = (126 \times 81 \times 9) \bmod 143 \quad C = (126 \times 81 \times 9) \bmod 143$$

This gives:

$$C = 48 \quad \boxed{C = 48} \quad C = 48$$

Therefore, the encrypted ciphertext is:

$$48 \quad \boxed{48} \quad 48$$

d. Decrypt and Verify

RSA decryption formula:

$$M = C \bmod n \quad M = C^d \bmod n \quad M = C \bmod n$$

$$M = 48 \bmod 143 \quad M = 48^{\{103\}} \bmod 143 \quad M = 48 \bmod 143$$

Using modular exponentiation:

$$M = 9 \quad \boxed{M = 9} \quad M = 9$$

Therefore:

$$\text{Decrypted Message} = 9 \quad \boxed{\text{Decrypted Message} = 9} \quad \text{Decrypted Message} = 9$$

The original message was also 9.

Hence:

$$M_{\text{original}} = M_{\text{decrypted}} = 9 \quad \boxed{M_{\text{original}} = M_{\text{decrypted}} = 9} \quad M_{\text{original}} = M_{\text{decrypted}} = 9$$

Verification successful.

e. Effect of Changing e on Security

The public exponent e influences RSA's computational performance and must satisfy:

$$1 < e < \phi(n) \quad 1 < e < \phi(n) \quad 1 < e < \phi(n)$$

and

$$\gcd(e, \phi(n)) = 1 \quad \gcd(e, \phi(n)) = 1 \quad \gcd(e, \phi(n)) = 1$$

A poorly selected or extremely small exponent can create security problems when RSA is implemented incorrectly, particularly without secure padding. Very large values of e increase encryption and signature-verification computation.

In practical RSA systems, a commonly used value is:

$$e = 65537$$

RSA security, however, depends mainly on sufficiently large keys, correct padding such as OAEP, secure key generation, and implementation—not simply choosing a larger e .

Result

$$n=143, \phi(n)=120, e=7, d=103 \quad n=143, \phi(n)=120, e=7, d=103 \quad n=143, \phi(n)=120, e=7, d=103$$

$$M=9 \rightarrow C=48 \rightarrow M=9 \quad M=9 \rightarrow C=48 \rightarrow M=9 \quad M=9 \rightarrow C=48 \rightarrow M=9$$

Q2. Block Cipher Mode Evaluation

Given

Plaintext blocks:

$$P = [1010, 1010, 1111, 1010] \quad P = [1010, 1010, 1111, 1010] \quad P = [1010, 1010, 1111, 1010]$$

Key:

$$K = 1100 \quad K = 1100 \quad K = 1100$$

CBC Initialization Vector:

$$IV = 0000 \quad IV = 0000 \quad IV = 0000$$

Since no specific block-cipher function is provided, for this educational calculation we assume the simplified encryption operation:

$$E_K(X) = X \oplus K \quad E_K(X) = X \oplus K \quad E_K(X) = X \oplus K$$

a. ECB Encryption

In Electronic Codebook mode, each plaintext block is independently encrypted.

$$C_i = P_i \oplus K \quad C_i = P_i \oplus K$$

Block 1

$$1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110$$

Block 2

$$1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110$$

Block 3

$$1111 \oplus 1100 = 0011 \quad 1111 \oplus 1100 = 0011 \quad 1111 \oplus 1100 = 0011$$

Block 4

$$1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110$$

Therefore:

$$CECB = [0110, 0110, 0011, 0110] \quad \boxed{C_{ECB} = [0110, 0110, 0011, 0110]} \quad CECB = [0110, 0110, 0011, 0110]$$

Notice that identical plaintext **1010** repeatedly produces ciphertext **0110**.

CBC Encryption

CBC first combines the plaintext with the previous ciphertext:

$$C_i = E_K(P_i \oplus C_{i-1}) \quad C_i = E_K(P_i \oplus C_{i-1}) \quad C_i = E_K(P_i \oplus C_{i-1})$$

where:

$$C_0 = IV = 0000 \quad C_0 = IV = 0000 \quad C_0 = IV = 0000$$

Block 1

$$P_1 \oplus IV = 1010 \oplus 0000 = 1010 \quad P_1 \oplus IV = 1010 \oplus 0000 = 1010 \quad P_1 \oplus IV = 1010 \oplus 0000 = 1010$$

Encrypt:

$$1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110 \quad 1010 \oplus 1100 = 0110$$

Therefore:

$$C_1 = 0110 \quad C_1 = 0110 \quad C_1 = 0110$$

Block 2

$$1010 \oplus 0110 = 1100 \quad 1010 \oplus 0110 = 1100 \quad 1010 \oplus 0110 = 1100$$

Encrypt:

$$1100 \oplus 1100 = 0000 \quad 1100 \oplus 1100 = 0000 \quad 1100 \oplus 1100 = 0000$$

Therefore:

$$C2 = 0000 \quad C_2 = 0000 \quad C2 = 0000$$

Block 3

$$1111 \oplus 0000 = 1111 \quad 1111 \oplus 0000 = 1111 \quad 1111 \oplus 0000 = 1111$$

Encrypt:

$$1111 \oplus 1100 = 0011 \quad 1111 \oplus 1100 = 0011 \quad 1111 \oplus 1100 = 0011$$

Therefore:

$$C3 = 0011 \quad C_3 = 0011 \quad C3 = 0011$$

Block 4

$$1010 \oplus 0011 = 1001 \quad 1010 \oplus 0011 = 1001 \quad 1010 \oplus 0011 = 1001$$

Encrypt:

$$1001 \oplus 1100 = 0101 \quad 1001 \oplus 1100 = 0101 \quad 1001 \oplus 1100 = 0101$$

Therefore:

$$C4 = 0101 \quad C_4 = 0101 \quad C4 = 0101$$

Thus:

$$CCBC = [0110, 0000, 0011, 0101] \quad \boxed{C_{\{CBC\}} = [0110, 0000, 0011, 0101]} \quad CCBC = [0110, 0000, 011, 0101]$$

b. Comparison of Outputs

Block	Plaintext	ECB	CBC
P1	1010	0110	0110
P2	1010	0110	0000

P3 1111 0011 0011

P4 1010 0110 0101

ECB produces the same output whenever the same plaintext block occurs.

CBC chains the blocks together, so the encryption of a block depends on previous ciphertext.

c. Which Mode Hides Patterns Better?

CBC hides patterns better than ECB\boxed{\text{CBC hides patterns better than ECB}}CBC hides patterns better than ECB

ECB independently encrypts blocks:

$P_1 = P_2 \Rightarrow C_1 = C_2$ $P_1 = P_2 \Rightarrow C_1 = C_2$

Therefore, repeated patterns in plaintext can remain visible in ciphertext.

In CBC:

$C_i = EK(P_i \oplus C_{i-1})$ $C_i = EK(P_i \oplus C_{i-1})$

Even when plaintext blocks are identical, their inputs to the block cipher generally differ because they are combined with different previous ciphertext blocks.

Result

ECB:

0110, 0110, 0011, 0110\boxed{0110,\,;0110,\,;0011,\,;0110}0110,0110,0011,0110

CBC:

0110, 0000, 0011, 0101\boxed{0110,\,;0000,\,;0011,\,;0101}0110,0000,0011,0101

Hence, CBC provides better pattern hiding than ECB. In real modern systems, authenticated-encryption modes such as AES-GCM are generally preferred.

Q3. DES vs 3-DES vs Blowfish Performance Study

Given

Algorithm	Key Size	Time per Block
DES	56 bits	2 ms
3-DES	168 bits	6 ms
Blowfish	128 bits	1 ms

Total number of blocks:

$N=1000$

i. Total Encryption Time

Formula:

$$T = N \times T_{\text{block}} \quad T = N \times T_{\text{block}} \quad T = N \times T_{\text{block}}$$

DES

$$T = 1000 \times 2 \text{ ms} \quad T = 1000 \times 2 \text{ ms} \quad T = 1000 \times 2 \text{ ms} = 2000 \text{ ms} = 2000 \text{ ms} = 2000 \text{ ms}$$

Since:

$$1000 \text{ ms} = 1 \text{ s} \quad 1000 \text{ ms} = 1 \text{ s} \quad 1000 \text{ ms} = 1 \text{ s} \quad T_{\text{DES}} = 2 \text{ seconds} \quad \boxed{T_{\text{DES}} = 2 \text{ seconds}} \quad T_{\text{DES}} = 2 \text{ seconds}$$

3-DES

$$T = 1000 \times 6 \text{ ms} \quad T = 1000 \times 6 \text{ ms} \quad T = 1000 \times 6 \text{ ms} = 6000 \text{ ms} = 6000 \text{ ms} = 6000 \text{ ms} \quad T_{\text{3DES}} = 6 \text{ seconds} \quad \boxed{T_{\text{3DES}} = 6 \text{ seconds}} \quad T_{\text{3DES}} = 6 \text{ seconds}$$

Blowfish

$$T = 1000 \times 1 \text{ ms} \quad T = 1000 \times 1 \text{ ms} \quad T = 1000 \times 1 \text{ ms} = 1000 \text{ ms} = 1000 \text{ ms} = 1000 \text{ ms} \quad T_{\text{Blowfish}} = 1 \text{ second} \quad \boxed{T_{\text{Blowfish}} = 1 \text{ second}} \quad T_{\text{Blowfish}} = 1 \text{ second}$$

Comparison

Algorithm	Total Time	Blocks/Second
DES	2 sec	500
3-DES	6 sec	≈ 166.7

Blowfish 1 sec 1000

Therefore, according to the given data:

Blowfish>DES>3DES\boxed{Blowfish>DES>3DES}Blowfish>DES>3DES

in encryption speed.

ii. Security and Performance Trade-offs

DES

DES is faster than 3-DES but has only a **56-bit effective key**, which is too small for modern security requirements and is vulnerable to brute-force attacks.

3-DES

3-DES applies DES multiple times and therefore provides substantially greater security than DES, but it is considerably slower.

Given:

6ms/block6ms/block6ms/block

it requires six seconds to process 1000 blocks.

It is also a legacy algorithm and is no longer a good choice for new systems.

Blowfish

Blowfish has a larger key size in the given scenario:

128 bits128\ bits128 bits

and provides the best performance:

1ms/block1ms/block1ms/block

Thus, among the **three options supplied in the question**, Blowfish provides the best balance of the stated key size and processing speed.

iii. Recommended Algorithm

Based strictly on the three given choices:

Blowfish\boxed{\text{Blowfish}}Blowfish

is the preferred option for the stated real-time communication scenario because it takes only:

1 second/1000 blocks\boxed{1\ second/1000\ blocks}1 second/1000 blocks

while DES requires 2 seconds and 3-DES requires 6 seconds.

For a newly designed real-world system, however, a modern standard such as **AES**, preferably with authenticated encryption such as AES-GCM, would normally be selected instead of these legacy ciphers.

Result

DES=2s\boxed{DES=2s}DES=2s 3DES=6s\boxed{3DES=6s}3DES=6s

Blowfish=1s\boxed{Blowfish=1s}Blowfish=1s

Among the given algorithms, **Blowfish offers the best stated performance-security trade-off.**

Q4. Diffie-Hellman Key Exchange Analysis

Given

$p=23$ $p=23$ $p=23$ $g=5$ $g=5$ $g=5$

User A private key:

$a=6$ $a=6$ $a=6$

User B private key:

$b=15$ $b=15$ $b=15$

d. Compute Public Keys

User A

Formula:

$$\begin{aligned}
 A &= g \bmod p \quad A = g^a \bmod p \quad A = g \bmod p \quad A = 56 \bmod 23 \quad A = 5^6 \bmod 23 \quad A = 56 \bmod 23 \\
 52 &= 25 \bmod 23 = 25^2 = 25 \bmod 23 = 252 = 25 \bmod 23 = 2 \quad 54 = 22 = 45^4 = 2^2 = 454 = 22 = 4 \\
 56 &= 54 \times 525^6 = 5^4 \times 5^2 = 54 \times 52 = 4 \times 2 = 8 = 4 \times 2 = 8
 \end{aligned}$$

Therefore:

$$A = 8 \quad \boxed{A=8} \quad A=8$$

User B

$$B = g \bmod p \quad B = g^b \bmod p \quad B = g \bmod p \quad B = 515 \bmod 23 \quad B = 5^{15} \bmod 23 \quad B = 515 \bmod 23$$

Using modular exponentiation:

$$\begin{aligned}
 52 \bmod 23 &= 25^2 \bmod 23 = 252 \bmod 23 = 2 \quad 54 \bmod 23 = 45^4 \bmod 23 = 454 \bmod 23 = 4 \\
 58 \bmod 23 &= 165^8 \bmod 23 = 1658 \bmod 23 = 16
 \end{aligned}$$

Since:

$$\begin{aligned}
 15 &= 8 + 4 + 2 + 115 = 8 + 4 + 2 + 115 = 8 + 4 + 2 + 1 \\
 515 &= 58 \times 54 \times 52 \times 55^{15} = 5^8 \times 5^4 \times 5^2 \times 5515 = 58 \times 54 \times 52 \times 5 \\
 &= (16 \times 4 \times 2 \times 5) \bmod 23 = (16 \times 4 \times 2 \times 5) \bmod 23 = (16 \times 4 \times 2 \times 5) \bmod 23 \\
 B &= 19 \quad \boxed{B=19} \quad B=19
 \end{aligned}$$

Thus:

$$A=8, B=19 \quad \boxed{A=8, \quad B=19} \quad A=8, B=19$$

e. Compute Shared Secret Key

User A calculates:

$$\begin{aligned}
 KA &= B \bmod p \quad K_A = B^a \bmod p \quad KA = B \bmod p \\
 KA &= 196 \bmod 23 \quad K_A = 19^6 \bmod 23 \quad KA = 196 \bmod 23 \quad KA = 2 \quad \boxed{K_A=2} \quad KA=2
 \end{aligned}$$

User B calculates:

$$\begin{aligned}
 KB &= A \bmod p \quad K_B = A^b \bmod p \quad KB = A \bmod p \\
 KB &= 815 \bmod 23 \quad K_B = 8^{15} \bmod 23 \quad KB = 815 \bmod 23 \quad KB = 2 \quad \boxed{K_B=2} \quad KB=2
 \end{aligned}$$

Therefore:

$$KA = KB = 2 \quad \boxed{K_A = K_B = 2} \quad KA = KB = 2$$

The shared secret is:

$$K = 2 \quad \boxed{K=2} \quad K=2$$

f. Man-in-the-Middle Attack

Basic Diffie-Hellman establishes a shared secret but does not by itself authenticate the communicating users.

Assume attacker **Eve** is positioned between A and B.

Normal communication:

$A \rightarrow B: A_{\text{public}}$
 $B \rightarrow A: B_{\text{public}}$

During MITM:

$A \rightarrow \text{Eve}$
 $\text{Eve} \rightarrow B$

Eve intercepts A's public key and sends her own public key to B.

Similarly, Eve intercepts B's public key and replaces it with another attacker-controlled public key before sending it to A.

The result becomes:

$A \leftrightarrow \text{Eve}$

and

$\text{Eve} \leftrightarrow B$

instead of:

$A \leftrightarrow B$

A may therefore establish one secret with Eve while B establishes another secret with Eve. Eve can potentially decrypt, modify, re-encrypt, and forward messages.

g. Prevention

A Man-in-the-Middle attack can be prevented by **authenticating the Diffie-Hellman exchange**.

Suitable mechanisms include:

- Digital signatures
- Digital certificates

- Public Key Infrastructure (PKI)
- Authenticated key-exchange protocols
- TLS using properly validated certificates

For example, the public Diffie-Hellman parameters can be digitally signed so that an attacker cannot replace them without detection.

Result

$A_{\text{public}}=8$ $B_{\text{public}}=19$
 Shared Secret=2

Diffie-Hellman securely establishes a shared secret against passive eavesdropping when appropriate parameters are used, but authentication is necessary to prevent MITM attacks.

Q5. ECC vs RSA Comparison

Given

RSA key size:

1024 bits

ECC key size:

160 bits

The question states these as having equivalent security for the purpose of the comparison.

Device:

Resource-constrained IoT device

d. Computational Cost

RSA relies on modular exponentiation with large integers.

With the given RSA key:

1024 bits

the device must perform arithmetic involving comparatively large operands.

ECC uses elliptic-curve point operations and, according to the question's assumed security equivalence, requires only:

160 bits

Therefore, ECC can achieve the assumed security level using much smaller key material.

The key-size difference is:

$1024 - 160 = 864$ bits

and the key-size ratio is:

$\frac{1024}{160} = 6.4$

So, in this simplified comparison, the RSA key is:

$6.4 \times$

the size of the ECC key.

e. Power and Memory Consumption

IoT devices generally have limited:

- RAM
- Storage
- Battery capacity
- CPU capability
- Network bandwidth

Smaller cryptographic keys can reduce storage and communication overhead.

RSA

RSA requires larger keys and signatures/ciphertexts depending on the operation, increasing memory, bandwidth, and computational requirements.

ECC

ECC provides strong security with significantly smaller key sizes in comparable-security settings.

This can reduce:

Memory Usage Communication

Overhead\text{Communication Overhead}Communication Overhead

and often:

Energy Consumption\text{Energy Consumption}Energy Consumption

for constrained devices.

Therefore:

ECC is generally more suitable for constrained IoT devices\boxed{\text{ECC is generally more suitable for constrained IoT devices}}ECC is generally more suitable for constrained IoT devices

f. Recommended Cryptosystem

For the scenario given, the recommended cryptosystem is:

ECC\boxed{\text{ECC}}ECC

Comparison

Parameter	RSA	ECC
Given key size	1024 bits	160 bits
Key storage	Higher	Lower
Communication overhead	Higher	Lower
Suitability for constrained IoT	Lower	Higher
Resource efficiency	Lower	Higher

ECC is suitable for IoT devices because its smaller keys reduce storage and communication requirements while supporting public-key operations such as authentication and key agreement.

Important Security Note

The **1024-bit RSA / 160-bit ECC equivalence is a historical textbook comparison and should not be interpreted as a recommendation for a new real-world system**. Both parameter sizes are below common modern security targets.

For the analytical problem, however, using the values supplied:

ECC is the better choice\boxed{\text{ECC is the better choice}}ECC is the better choice

because the resource-constrained IoT device benefits from smaller keys and reduced

memory and communication overhead.

Overall Conclusion

The analytical problems demonstrate the importance of selecting cryptographic algorithms according to security requirements, computational resources, and application environment.

RSA demonstrates public-key encryption and the relationship between public and private keys. **CBC** provides better pattern hiding than ECB. Among the legacy symmetric algorithms specified in the problem, **Blowfish** provides the fastest stated processing time.

Diffie-Hellman enables two parties to derive a shared secret but requires authentication to resist MITM attacks. Finally, **ECC** is generally advantageous for constrained IoT environments because comparable security can be achieved with smaller keys than RSA.

Thus, modern information security requires not only strong cryptographic algorithms but also appropriate **key management, authentication, integrity protection, secure protocols, and correct implementation.**